

Guia Completo de Instalação e Execução: PixelForge Support

Este documento fornece as instruções passo a passo para configurar o ambiente, compilar e executar o projeto **PixelForge Support** localmente.

1. Preparação do Ambiente (Pré-requisitos)

Antes de começar, o computador onde o projeto será testado precisa de ter as seguintes ferramentas instaladas:

- **Git:** Para transferir o código do repositório.
- **Flutter SDK (versão 3.13.5 ou superior):** O *framework* principal. Certifique-se de que o comando `flutter doctor` não apresenta erros críticos.
- **Editor de Código:** Recomendado o Visual Studio Code (com as extensões de Flutter e Dart instaladas) ou o Android Studio.
- **Dispositivo de Teste:** Um emulador Android a correr no computador ou um telemóvel físico com o "Modo de Programador" e a "Depuração USB" ativados.

2. Transferir o Código Fonte

Abra o seu terminal (Prompt de Comando, PowerShell ou Terminal do Mac/Linux) e execute o comando abaixo para clonar o repositório para a sua máquina local:

```
Bash
git clone https://github.com/SEU_USUARIO/pixelforge-support.git
```

(Nota: Substitua `SEU_USUARIO` pelo seu nome de utilizador do GitHub).

Após a transferência ser concluída, navegue para dentro da pasta do projeto:

```
Bash
cd pixelforge-support
```

3. Descarregar as Dependências do Projeto

O projeto utiliza pacotes externos que não são enviados para o GitHub para poupar espaço. Para descarregar os pacotes necessários (como o sistema HTTP nativo e o leitor de variáveis de ambiente), execute:

```
Bash
flutter pub get
```

Aguarde alguns segundos até que o terminal confirme que todas as dependências foram resolvidas.

4. Configurar a Inteligência Artificial (Ficheiro .env)

Por razões de segurança e boas práticas de engenharia de software, a chave de acesso à API do Google Gemini não é partilhada no código público. Para que o *bot* de IA funcione, é estritamente necessário criar um ficheiro de configuração local.

1. Aceda à plataforma [Google AI Studio](#) e inicie sessão com uma conta Google.
2. Clique em **"Create API Key"** e copie a chave gerada.
3. Na pasta raiz do projeto (exatamente no mesmo nível onde se encontra o ficheiro `pubspec.yaml`), crie um ficheiro novo e dê-lhe o nome de `.env` (certifique-se de que inclui o ponto inicial).
4. Abra o ficheiro `.env` e cole a variável com a sua chave, sem utilizar aspas, com a seguinte estrutura:

Snippet de código

```
GEMINI_API_KEY=sua_chave_copiada_aqui
```

5. Guarde o ficheiro. O pacote `flutter_dotenv` encarregar-se-á de ler este ficheiro e injetar a chave de forma segura no serviço HTTP da aplicação.

5. Compilar e Executar a Aplicação

Com o dispositivo físico ligado por cabo (ou o emulador aberto), certifique-se de que o Flutter reconhece o dispositivo executando:

```
Bash  
flutter devices
```

Se o seu dispositivo aparecer na lista, inicie a compilação do projeto com o comando final:

```
Bash  
flutter run
```

Nota sobre a primeira execução: A primeira compilação para Android demora geralmente alguns minutos, pois o sistema *Gradle* precisará de descarregar ferramentas e estruturar o pacote APK. As execuções seguintes serão quase instantâneas através da funcionalidade *Hot Reload*.

6. Resolução de Problemas Comuns (Troubleshooting)

- **Erro ao compilar (Exit code 1):** Verifique se o seu dispositivo ou emulador possui o Android 5.0 (API nível 21) ou superior, que é o requisito mínimo do SDK definido neste projeto.
- **A IA não responde (Erro 503):** O modelo `gemini-flash-latest` pode estar a passar por um pico de tráfego. A aplicação já possui um sistema de *retry* automático, mas se o problema persistir, aguarde uns minutos e envie a mensagem novamente.
- **Erro na chave de API:** Se a aplicação reportar erro de ligação imediatamente, confirme se o ficheiro `.env` está corretamente nomeado, se está na pasta raiz e se a chave colada é válida e está ativa no painel do Google.